混合式Transformer模型下计算机软件代码缺陷自动修复

任占军
（青海西宁市湟中区职业教育中心，西宁 730207）

摘　要：现代软件系统常采用非统一内存访问架构，使得代码缺陷呈现跨模块传播、时序敏感等复杂特征。仅依赖语义特征构建修复模型，难以捕捉此类缺陷引发的语义-结构断层，导致代码缺陷修复成功率较低。基于此提出了混合式Transformer模型下计算机软件代码缺陷自动修复算法，构建代码-缺陷传播图，引入结构感知的加权相似度度量，融合缺陷传播因子与缓存一致性特征，实现跨层级缺陷特征的精准捕获。研究构建了代码缺陷修复模型，将语义与度量特征动态加权融合，经残差连接输入混合式Transformer模型，分两个阶段优化补丁生成分支和上下文建模，用混合模型解析结构特征，结合修复模型提取依赖关系，生成候选修复序列，引入动态束搜索策略以增强多样性，通过筛选输出多个修复方案，在训练过程中采用奖励函数引导生成合理代码，实现计算机软件代码缺陷的自动修复。

关键词：Transformer模型；计算机软件；代码缺陷；余弦相似度；自动修复
DOI:10.16184/j.cnki.comprg.2026.05.014

1　概述

在软件工程领域，实现代码缺陷的自动修复始终是提升开发效率及软件可靠性的核心研究方向。随着软件系统规模与复杂度的指数级增长，在异构计算架构与深度优化编译环境下，由非统一内存访问、指令级并行等因素引发的代码缺陷呈现跨模块传播、时序敏感等新型特征。因为传统人工修复方式面临成本高昂、周期冗长、易引入新缺陷等挑战，所以亟须发展更智能的自动化修复技术。

在现有修复方法中，曹鹤玲等[1]引入Transformer架构以增强代码语义理解能力，虽能通过自注意力机制捕捉长距离依赖关系，但在修复循环错误的缺陷时，准确性有待提升。薄莉莉等[2]从历史修复数据中提取模板构建规则库，在特定缺陷模式下表现稳定，但模板覆盖范围有限，难以适配动态变化的代码上下文，会生成无效修复。许鹏宇等[3]用预训练语言模型捕捉代码统计规律，但存在生成结果不可控问题，难以保证语义正确性。常建明等[4]构建了面向缺陷定位的代码搜索引擎，通过AST匹配相似缺陷模式，虽可实现快速定位，但该方法无法捕捉运行时行为，在修复并发竞争等动态缺陷时，定位结果与实际执行路径存在偏差。

现有方法在面对由底层硬件特征引发的复杂缺陷时修复效果难以达到实际要求，故提出混合式Transformer模型下计算机软件代码缺陷自动修复算法。通过构建代码-缺陷传播图显式建模跨层级缺陷特征，结合多模态特征融合机制协同分析语义与结构信息，并采用分阶段优化策略提升修复准确性，有效应对现代软件系统中日益复杂的缺陷修复挑战。

2　基于结构感知相似度的跨层级代码缺陷特征捕捉

在处理计算机软件代码缺陷特征相似度提取任务时，首先，对缺陷报告中的代码片段及关联注释进行结构化预处理，包括统一语法格式、去除特殊符号与多余空格、拆分多行代码，并将其转换为小写形式。然后，把处理后的文本转成特征向量矩阵。

为量化不同层级代码缺陷的语义关联性，研究通过构建代码-缺陷异构图，将代码语句作为节点，将缺陷关联关系作为边[5]，生成低维稠密特征向量，捕捉这种跨层级的语义关联，其数学表达如公式（1）所示：

$$h_i^{l+1}=\sum \alpha_i^l \boldsymbol{W}\cdot \boldsymbol{h}_i^l \tag{1}$$

其中，$\boldsymbol{h}_i^l$为异构图第l层节点i的特征向量；α_i^l为注意力权重；$\boldsymbol{W}$为可训练参数矩阵。

在特征匹配阶段，采用改进的余弦相似度度量方法，引入缺陷类型权重因子β与代码结构复杂度调节项γ，构建多层级代码缺陷结构感知的相似度评估函数，如公式（2）所示：

$$S(\boldsymbol{X},\boldsymbol{Y})=\beta\frac{\boldsymbol{X}\cdot\boldsymbol{Y}}{\|\boldsymbol{X}\|\ \|\boldsymbol{Y}\|}+\gamma\cdot h_l^{l+1}(\boldsymbol{X},\boldsymbol{Y}) \tag{2}$$

其中，$\boldsymbol{X}$为语义特征向量；$\boldsymbol{Y}$为缺陷模式度量向量。

依据相似度得分进行排序，输出与当前缺陷最为匹配的特征，有效提高了缺陷特征匹配的精准度。

3　基于混合式Transformer模型的代码缺陷修复模型构建

研究采用混合式Transformer模型，构建高精度的代

作者简介：任占军（1972—），男，本科，高级讲师，研究方向为计算机网络技术。

中国知网　https://www.cnki.net

码缺陷修复模型，通过模型中的CNN结构提取代码片段的语法特征，结合自注意力机制捕捉函数调用链、全局变量作用域等长距离依赖，从而更准确地定位复杂缺陷。

将代码片段的语义特征向量$\boldsymbol{X}=[x_1,x_2,\cdots,x_n]$与缺陷模式度量向量$\boldsymbol{Y}=[y_1,y_2,\cdots,y_m]$进行动态加权融合。通过引入自适应注意力机制，计算语义与度量特征的权重分配系数$\boldsymbol{\varpi}$，其计算过程如公式（3）所示：

$$\boldsymbol{\varpi}=\sigma[W_a(x_n,y_m)+b] \tag{3}$$

其中，σ为Sigmoid激活函数；W_a和b为可训练参数。

融合后的特征向量Z借助残差连接强化特征传递，其表达式如公式（4）所示：

$$\boldsymbol{Z}=\boldsymbol{\varpi}\cdot x_n+(1-\boldsymbol{\varpi})y_m+x_n \tag{4}$$

将融合特征$\boldsymbol{Z}$输入混合式Transformer模型[6]后，代码缺陷修复模型的构建分为两个阶段。

阶段1采用小批量梯度下降法，通过交叉熵损失函数优化补丁生成分支，其公式定义如公式（5）所示：

$$L_{\mathrm{CE}}=-\frac{1}{Z}\sum_{j=1}^{N}\sum_{k=1}^{K}y_{jk}\log\bar{y}_{jk} \tag{5}$$

其中，N为样本数；K为类别数；y_{jk}和$\bar{y}_{jk}$分别为第j个样本属于k类别的真实标签与预测概率。此阶段意在验证模型对代码语法结构的解析能力。

阶段2引入上下文建模，进而得出编码代码的依赖关系。节点p的上下文表示S_p由前向S_{pq}与后向S_{ph}状态拼接而成，其计算过程如公式（6）所示：

$$\boldsymbol{S}_p=L_{\mathrm{CE}}\left(S_{pq}\oplus S_{ph}\right) \tag{6}$$

最终生成的上下文矩阵$\boldsymbol{S}_p$为缺陷模式判别提供全局语义支持。

4 计算机软件代码缺陷自动修复

研究通过混合式Transformer模型解析代码结构，结合修复模型提取变量等依赖关系，输入解码器生成候选序列，分层优化后输出字符概率分布[7]。引入动态束搜索策略，设时间步t的候选集合为B_t，其规模g_t动态计算如公式（7）所示：

$$g_t=\min\left[g_{\max},\frac{|B_{t-1}|\cdot\exp(-\lambda,L_{t-1})}{\sum\exp(-\lambda,L_t)}\right] \tag{7}$$

其中，$g_{\max}$为最大时间规模；λ为衰减系数；L_t为候选序列在时间步t的累积损失。

在计算机软件代码缺陷自动修复阶段，每个候选序列生成V个可能字符（V为词汇表大小），形成规模为$g_t\cdot V$的扩展集R_t。筛选最优子集的公式如公式（8）所示：

$$R_{t+1}=\arg\max_{B_t=g_{t+1}}\sum_{B_t}\log\varphi(B_t|g_t\cdot V) \tag{8}$$

其中，$\varphi(B_t|g_t\cdot V)$为序列B_t的条件概率。

该过程持续到生成终止符或达最大长度为止，输出g_{t+1}个修复方案。训练时用奖励函数引导，大幅提升复杂缺陷场景的修复成功率。

5 实验结果与分析

5.1 实验环境设置

实验的硬件采用AMD Ryzen Threadripper 3990X处理器、256 GB内存，采用Ubuntu 20.04 LTS系统，配备2 TB双NVMe SSD RAID0阵列作为存储。实验环境配置了分布式计算框架，利用4台GPU服务器构建异构计算集群，节点间通过InfiniBand HDR网络实现低延迟通信。开发环境遵循工业级标准，将Python版本升级至3.8.5，采用PyTorch 1.12.1搭配CUDA 11.6驱动，集成NCCL 2.12.12多机通信库。代码仓库采用GitLab企业版管理，采用Jenkins自动构建持续集成，采用IPMI实时监控硬件参数，以保证实验可复现。

5.2 实验准备

在实验准备阶段，需配置混合式Transformer模型的超参数，如表1所示。表1展示了训练的核心超参数。学习率衰减采用余弦退火策略，初始衰减率为0.85，参数初始化使用Kaiming Normal分布。

表1 混合式Transformer模型训练核心超参数配置

参数类别	参数项	具体配置
基础学习参数	初始学习率	0.00015
优化器	优化器类型	AdamW（β_1=0.9, β_2=0.999）
学习率调度	衰减策略	余弦退火
批次配置	Batch Size	64
正则化	Dropout率	0.15（Transformer层间） 0.3（全连接层）
嵌入维度	词向量维度	300
梯度裁剪	最大梯度范数	1.0
训练周期	最大Epoch数	12 000

实验采用双数据集并行训练策略，其中，主数据集为Java+JML与SyntheticBugs的混合补丁集，包含412个Java+JML修复样本及317个合成缺陷样本。辅助数据集为Defects4J的扩展版本，新增156个跨项目缺陷修复案例。

将数据集按照7∶2∶1比例划分为训练集、验证集和测试集，并通过分层抽样确保各类缺陷类型在各子集中均匀分布，各计算机软件代码缺陷类型的部分代码如图1所示。

中国知网 https://www.cnki.net

```
class Node:
2 def__init__(self):
3 self.parent = None
4 self.children = []
5 defadd_child(self, child):
6 self.children.append(child)
7 child.parent = self #
```

（a）内存泄漏缺陷

```
def faulty_factorial(n):
2 result = 1
3 for i in range(0, n): # 错误起点：从0开始
4 result *= i
5 return result
```

（b）逻辑错误缺陷

```
import threading
2 counter = 0
3
4 def increment():
5 global counter
6 for _ in range(1000000):
7 counter += 1 # 原子性破坏点
```

（c）并发竞争缺陷

图1　各计算机软件代码缺陷类型的部分代码

通过静态分析工具提取了89 214个测试用例的切片特征，其中，32%的样本存在跨文件调用关系，所有数据集均已脱敏，符合GDPR规范。

为验证设计的基于混合式Transformer模型的计算机软件代码缺陷自动修复算法的有效性，选取文献[1]方法、文献[2]方法、文献[3]方法和文献[4]方法作为对比方法，采用模型损失曲线和修复成功率为指标。

5.3　实验结果分析

采用文献[1]~文献[4]与所提算法的模型进行训练，随着训练轮次增加，5种模型的损失曲线结果如图2所示。

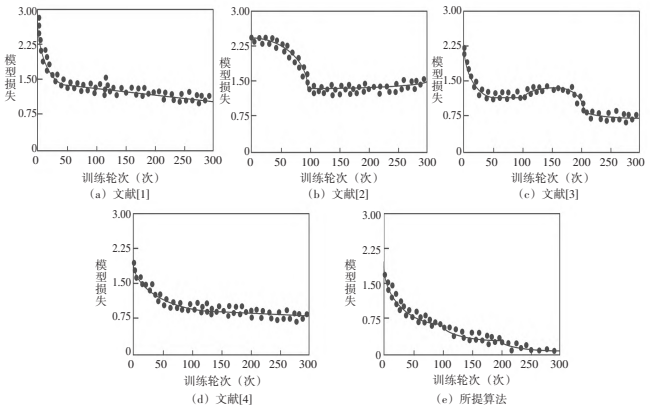


图2　5种模型的损失曲线结果

由图2可知，所提方法在0~100轮的训练初期，损失值由初始高位快速降至0.72，明显优于文献[1]的1.45与文献[4]的0.89。在200轮后的训练中后期，方法的损失稳定收敛于0.41，未出现文献[3]中的震荡现象，说明所提模型具备良好的训练稳定性与抗过拟合能力。并且所提方法在250轮后达到最优收敛状态，损失值较初始下降85.7%，显著高于文献[1]的49.5%与文献[4]的60%，说明所提方法可以在复杂代码上下文中保持一致的优化方向，并快速理解并掌握代码修复的核心模式。

为了评估算法对不同类型代码缺陷的修复能力，选取8类典型代码缺陷，具体为内存泄漏M1、逻辑错误M2、并发竞争M3、空指针异常M4、数组越界M5、循环错误M6、API误用M7、资源未释放M8，采用所提算法与文献[1]~文献[4]算法进行代码缺陷修复，通过代码缺陷修复成功率指标进行评估，对比结果如图3所示。

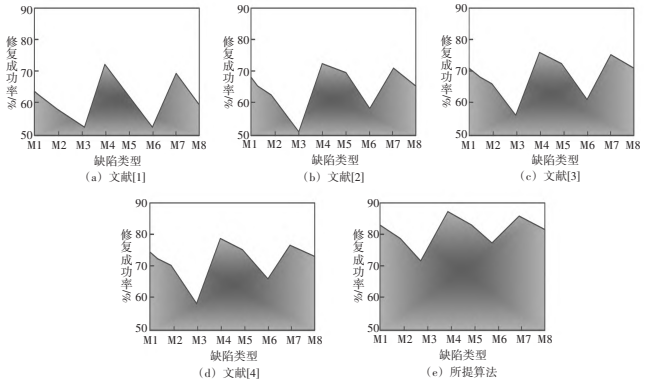


图3　代码缺陷修复成功率对比结果

由图3可知，在M1缺陷修复场景下，所提算法的成功率达到83.6%，较文献[4]提高12.7%，这表明研究所构建的代码–缺陷传播图能有效捕捉资源生命周期相关的依赖关系。针对M3高复杂度缺陷，成功率由62.8%提升至72.1%，增幅达到20.6%，充分体现了混合式Transformer模型在建模时序敏感型缺陷方面的优势，其双通路注意力机制能够协同分析线程交互模式与共享状态变迁，从而生成符合并发语义的修复补丁。在M8缺陷修复中，通过结合静态与动态方法，成功率达到81.8%，有效解决了传统方法的盲区问题。综合8类缺陷，所提算法平均成功率为80.0%，比文献[1]~ 文献[4]最高值高11.4%，各类型修复效果最优。上述结果说明，所提算法可以根据缺陷类型的特性，灵活选择和组合方法，更有效地对整个项目的代码缺陷进行修复，从而提高软件质量。

6　结语

研究提出了混合式Transformer模型下计算机软件代码缺陷自动修复算法，利用自注意力机制捕捉长距离依赖关系，并结合图神经网络强化局部分析能力，同时采用动态束搜索策略来平衡效率与多样性。在内存泄漏等典型缺陷场景中，该算法的修复成功率显著提升，能够生成多样且规范的修复方案。未来，该算法将结合静态与动态分析来降低误报率，并探索轻量化部署方案，以推动其向智能开发伙伴的方向发展。

参考文献

[1]　曹鹤玲，刘昱，韩栋．基于自注意力机制神经机器
（下转第56页）

中国知网　https://www.cnki.net

```
            m_tracer->position->setCoords(xClk, (*y).
at(xClk));
             m_label->setText(QString("% 1,% 2"). arg
(xClk).arg((*y).at(xClk)));
                 //标签显示于锚点上方
            m_label->position->setCoords(0,-10);
            m_tracer->setVisible(true);
            m_label->setVisible(true);
            LarPlot->replot();
        }
    }
  }
  return QObject::eventFilter(obj,ev);
}
```

辅助分析波形如图3所示。

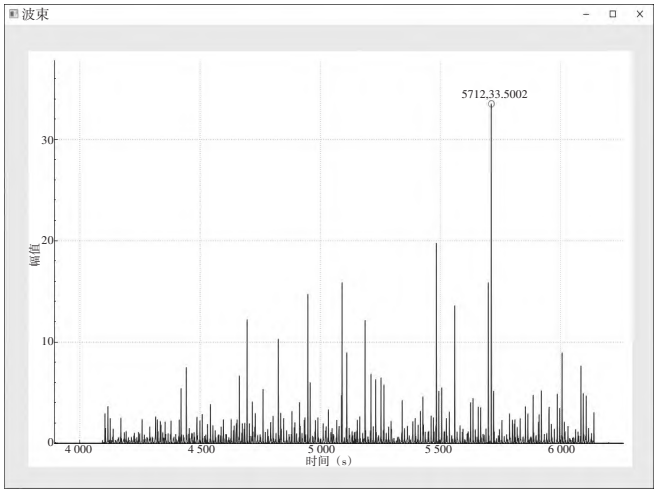


图3 辅助分析波形

4.3 数据结构设计

显示软件中的数据源自网络接收的实时数据，经解析处理后输出显示，为保证程序运行流畅与数据的完整性，采用多线程机制，除网络接收线程外，为每个显示窗口开辟一个线程[2]。采用QVector容器作为显示软件数据存储的基本结构，然而，QVector为提供内置的线程安全保障，在多线程并行读写时易发生内存访问冲突。为弥补这一缺陷，显示软件创建LockVector类，派生自QVector类，封装QReadWriteLock读写锁实现数据容器的线程保护。选择QReadWriteLock的原因是相较互斥锁QMutex，读写锁能够进行多线程读取，在保证线程安全的同时能够提升数据吞吐量，保证软件数据的实时性，更适合显示软件。LockVector针对显示软件运行所需的接口进行封装，包括void append(QByteArray t)、void remove(int i, int n)、int size()、QByteArray takeFirst()，在原函数操作前后执行加锁和解锁操作，调用时无需再管理锁的状态。

5 结语

研究描述了基于Qt框架的通用浮标数据显示软件全流程设计与实现方法。显示软件依托QCustomPlot组件解决了多格式数据适配、自定义显示样式等关键问题，能够用于灵活观测多种浮标数据时域数据。同时，该软件也存在一定的局限性。（1）无法实现放大波形动态的动态显示，只能显示某一时刻的冻结波形图。（2）只能显示浮标时域数据，无法显示处理后的频域数据。后续将从这两方面进行软件迭代优化，进一步提升软件的通用性与实用性。

参考文献

[1] 徐瑶. Qt中基于QCustomPlot实现曲线绘制和显示的研究[J]. 科技视界, 2019(25): 54-55.

[2] 吴昊, 蒋培培. 基于国产化平台的QCustomPlot多图绘制性能提升方法[J]. 无线互联科技, 2025, 22(2): 121-124.

[3] 钟权, 沈静波, 路伟欣. 基于QCustomPlot和Qt的曲线绘制及显示技术[J]. 科技视界, 2017(6): 46-47.

[4] 钱一滔. 论TCP/IP协议和多线程通信软件的设计及实现研究[J]. 电子技术与软件工程, 2015(10): 55.

（上接第37页）

翻译的软件缺陷自动修复方法[J]. 电子学报, 2024, 52(3): 945-956.

[2] 薄莉莉, 朱轩锐, 孙小兵. 基于模板的软件缺陷修复推荐方法[J]. 小型微型计算机系统, 2022, 43(5): 1096-1101.

[3] 许鹏宇, 况博裕, 苏铓, 等. 基于大语言模型的自动代码修复综述[J]. 计算机研究与发展, 2025, 62(8): 2040-2057.

[4] 常建明, 薄莉莉, 孙小兵. 面向缺陷定位的代码搜索引擎[J]. 计算机科学, 2021, 48(12): 140-148.

[5] 郭佳霖, 智敏, 殷雁君, 等. 图像处理中CNN与视觉Transformer混合模型研究综述[J]. 计算机科学与探索, 2025, 19(1): 30-44.

[6] 刘楷正, 乔阳阳, 董涛, 等. 人工智能下复杂软件源代码缺陷精准校正[J]. 计算机仿真, 2023, 40(8): 389-392.

[7] 程靖云, 王布宏, 罗鹏. 基于深度语义融合的代码缺陷静态检测方法[J]. 计算机应用, 2022, 42(10): 3170-3176.

中国知网 https://www.cnki.net